[cmp653 > final > defense]

Privacy-Budget Consumption in Repeated Aggregate SQL Workloads

> a statistical model _

refik can öztaş // N25142279

hacettepe university | ankara

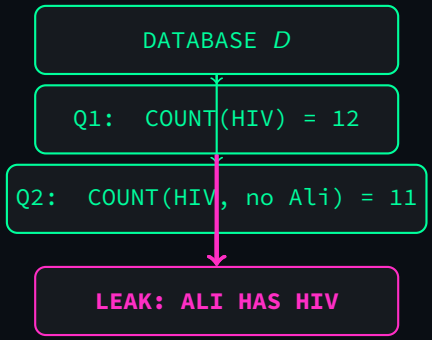
> 01 ::: aggregate queries are not safe

// intuition (wrong)
“aggregates return totals, not rows – safe.”

// reality
two overlapping COUNTs isolate one person.

```
- 12 patients with HIV
SELECT COUNT(*) FROM patients
WHERE zip='06897' AND
      disease='HIV';
- 11 with HIV, age <> 43
SELECT COUNT(*) FROM patients
WHERE zip='06897' AND
      disease='HIV' AND
      age <> 43;
```

⇒ the 43-year-old in 06897 has HIV.



Dinur & Nissim 2003:
O(n) noisy aggregates reconstruct the whole DB.

> 02 ::: differential privacy in 30 seconds

def: ε-differential privacy

A randomized mechanism $\mathcal{M}$ is ε-DP if for any two neighbouring databases $D \sim D'$ (differ by one row) and any output S:

$$\Pr[\mathcal{M}(D) \in S] \;\leq\; e^{\varepsilon} \cdot \Pr[\mathcal{M}(D') \in S]$$

// in practice: Laplace mechanism

$$\widetilde{f}(D) \;=\; f(D) + \eta, \qquad \eta \sim \mathsf{Lap}(\Delta f / \varepsilon)$$

// budget composition: ε accumulates over queries.

the engineering question

A finite budget $\varepsilon_{\text{total}}$ is consumed as the workload runs.

HOW does it degrade? Predicting this before deployment is what this project does.

> 03 ::: milestone version (what i had)

- Python middleware: SQL parser + sensitivity analyzer + Laplace mechanism
- Budget ledger with exact-repeat caching
- TPC-H benchmark, 4 workload families (W1–W4)
- 28 unit tests, working prototype

milestone headline

“Workload-aware caching saves 100× budget on a repetitive workload by exploiting DP’s post-processing property.”

// i thought this was the contribution.

> 04 ::: based on the review i received

// points the reviewer raised:

- caching's role in DP wasn't clear
- Algorithm 1 was too thin
- $1 - 1/k$ left unexplained
- AVG mechanism wasn't justified
- caching as a contribution is debatable

// and a structural ask:

"exact-repeat assumptions are strong.
budget and temporality should be considered together.
leakage / savings models are limited.

MECHANISM
(caching)

= debatable

MODEL
(workload structure → budget)

= contribution

// the framing of the contribution had to flip.

> 05 ::: research question, reframed

old: “does caching help?” // trivial

new:

Given a workload distribution and a privacy budget, can a closed-form statistical model predict the realized budget consumption and per-query utility before any query is issued?

Caching is now the corner case of the model, not the contribution.

// six concrete deliverables:

1. Analytical model (R2)
2. Temporal extension (R3)
3. Middleware + 62 unit tests
4. Predictive allocator (new mechanism)
5. Leakage analysis (R4)
6. 4,155-trial campaign (R6)

> 06 ::: the model in one slide

// setup: workload of k queries drawn i.i.d. from $\{p_i\}_{i=1}^{m}$.

prop 1 – expected unique queries

$$\mathbb{E}[u_k] \;=\; \sum_{i=1}^{m} \left[1 - (1 - p_i)^k\right]$$

(coupon-collector / occupancy on a non-uniform distribution)

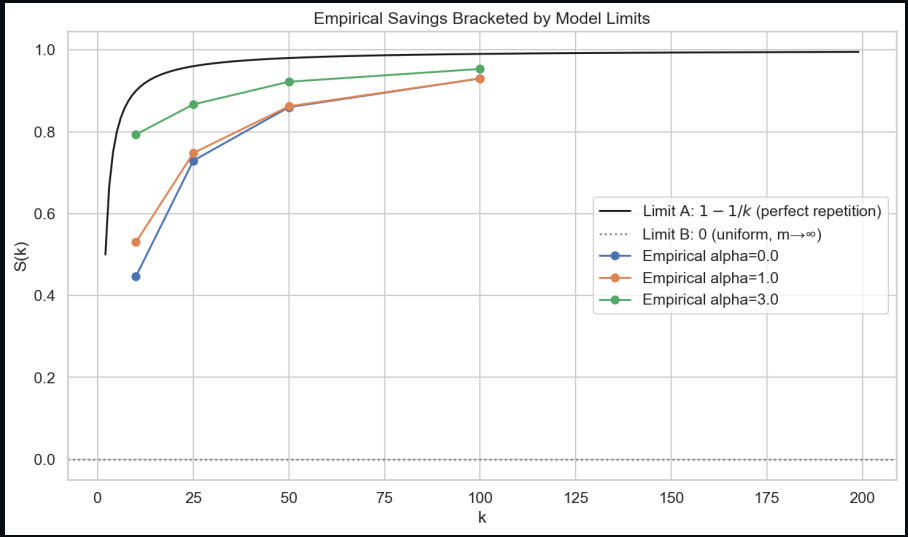
prop 2 – workload-aware budget

$$\mathbb{E}[\varepsilon_{\text{wa}}] \;=\; \varepsilon_q \cdot \mathbb{E}[u_k] \qquad S(k) \;=\; 1 - \tfrac{1}{k} \sum_{i} \left[1 - (1 - p_i)^k\right]$$

two limits:

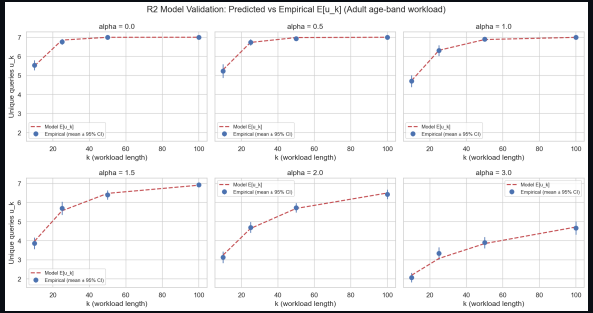
- $\alpha \to \infty$ (perfect repetition, $p_1 = 1$): $S(k) = 1 - 1/k$ // the milestone's toy formula

> 07 ::: the two limits, empirically



dashed: model prediction | markers: empirical mean over 30 trials

> 08 ::: main grid – 21 of 24 cells within 3%



// markers should land on the dashed identity

// setup

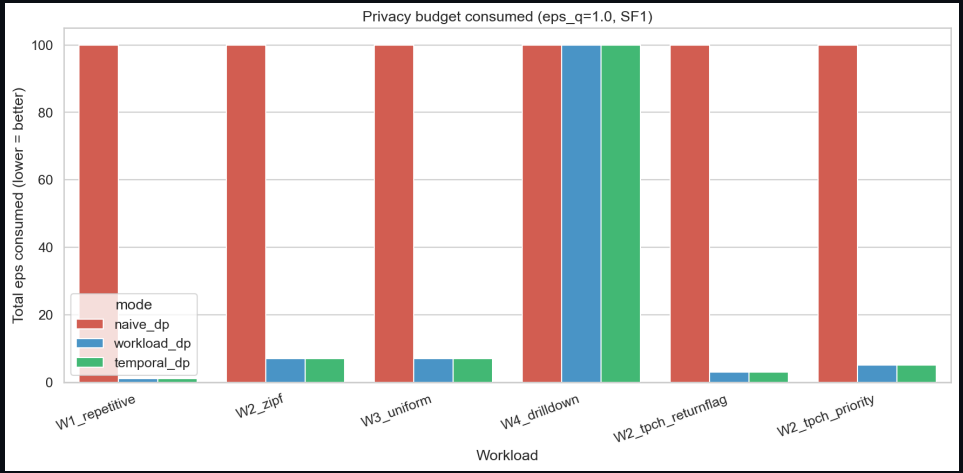
- $\alpha \in \{0, 0.5, 1, 1.5, 2, 3\}$
- $k \in \{10, 25, 50, 100\}$
- 30 trials per cell
- $\rightarrow$ 720 trials

// headline

- 21 / 24 cells within 3%
- Worst case 8.6% at small k, high α

Predict privacy cost from the workload distribution – before any query runs.

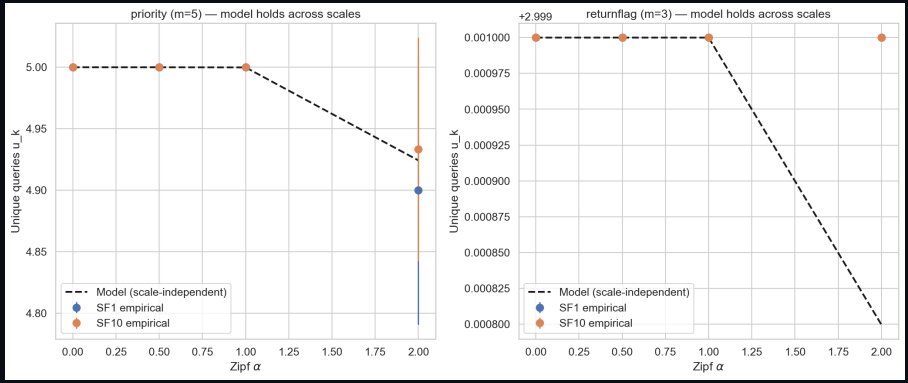
> 09 ::: full benchmark - 6 workloads x 3 modes



$\varepsilon_q = 1$, total budget $B = 100$, $k = 100$, 30 trials per cell.

red = naive (consumes full B) | blue = workload-DP | green = temporal-DP

> 10 ::: scale-independent (sf=1 vs sf=10)



TPC-H SF=1 (6M rows) vs SF=10 (60M rows)

Empirical $\mathbb{E}[u_k]$ identical within 0.03 units

$\Rightarrow$ the model captures workload structure, not data scale.

> 11 ::: heterogeneous workload – exact match

// W1 + W4 hybrid: fraction f repetitive, $1-f$ unique.
by construction, true $u_k = 1 + (1-f)\cdot k$.

f	true u_k	empirical ε	predicted $\varepsilon_q \cdot u_k$
0.1	91	45.5	45.5
0.3	71	35.5	35.5
0.5	51	25.5	25.5
0.7	31	15.5	15.5
0.9	11	5.5	5.5

⇒ empirical matches prediction to the cent in every cell.

// closed form $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \cdot u_k$ holds without requiring Zipf parametricity.

> 12 ::: predictive allocator - the new mechanism

// use the model's $\mathbb{E}[u_k]$ estimate at runtime to choose ε_q:

1. observe template frequencies online
2. estimate $\hat{U} = \mathbb{E}[u_k \mid \hat{p}]$ from Proposition 1
3. allocate $\varepsilon_q^{\text{pred}} = B/\hat{U}$

novel because

PINQ / PrivateSQL / Chorus / DOP-SQL all fix ε_q offline.
This is the first DP-SQL mechanism that adapts ε_q at runtime from a learned model.

B	Mode	MAE	vs naive
1	naive	101.1	–
	predictive	68.7	−32%
10	naive	10.1	–
	predictive	8.3	−18%
50	naive	2.0	–
	predictive	1.58	−21%

> 13 ::: ai/nlp bonus - semantic cache

// idea: two queries with similar ASTs might share a cache entry.

// two similarity signals:

- Tree Kernel (Collins-Duffy 2001) - shared subtree count, no training
- AST embedding (sentence-transformers / CodeBERT) - cosine on dense vector

honest negative result

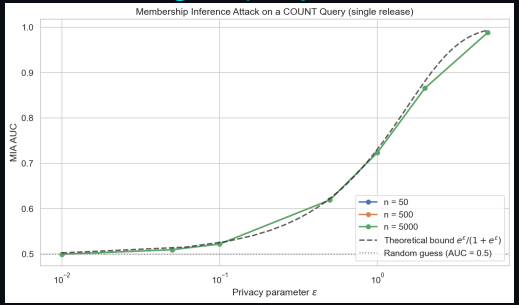
The semantic L2 cache fails in both directions:

- false positives: reuses cached noisy answer on queries with different true answers ⇒ wrong by thousands of count units
- false negatives: misses textbook equivalences (commutative AND, integer comparators, redundant predicates)

semantic similarity ≠ formal equivalence.
"AI-powered DP cache" needs a symbolic equivalence prover on top.

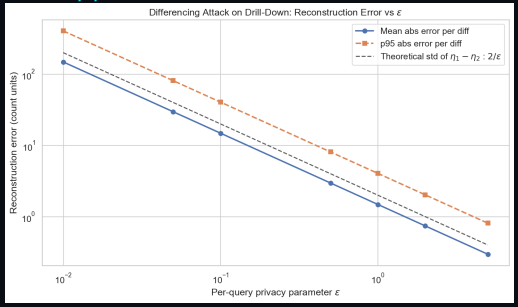
> 14 ::: privacy leakage, quantified

// single-query MIA AUC



empirical tracks theoretical $e^{\varepsilon}/(1+e^{\varepsilon})$ within 1%

// reconstruction error



differencing on 5-level drill-down, error $\sim \sqrt{2}/\varepsilon$

$\Rightarrow$ implementation matches the theory in the tight regimes.

> 15 ::: workload-level shadow MIA - a loose bound

logistic-regression classifier trained on simulated middleware outputs

AUC stays at 0.15-0.58 even though the cumulative-ε bound approaches 1.0

$\Rightarrow$ cumulative-ε is loose for realistic workloads.

> 16 ::: what we found – four takeaways

1. **Workload-aware budget is structural, not a cache trick.**
   $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \cdot \mathbb{E}[u_k]$ predicts within 3% on the main grid, **exact** on the mixed workload, same at SF=1 and SF=10.

2. **The $1 - 1/k$ toy rule is the $\alpha \to \infty$ corner case.**
   The model recovers it **and** the entire interior up to naive composition.

3. **Cumulative-ε overestimates real attack success.**
   Shadow MIA on W1–W4 stays at AUC 0.15–0.58 vs. a 1.0 cumulative bound – budget is over-provisioned in practice.

4. **Semantic similarity $\neq$ equivalence.**
   Tree-kernel + embedding admits false positives **and** misses textbook equivalences. “AI-powered DP” alone is not DP-safe.

> 17 ::: limitations + future work

// what this is not:

- ▶ Not a head-to-head reimplementation of PINQ / PrivateSQL / Chorus / DOP-SQL
- ▶ Not validated on a real workload trace (Zipf is a controlled sweep)
- ▶ Not a new cryptographic privacy mechanism

// concrete next steps:

- ▶ Cache re-release on budget surplus
- ▶ Rényi DP / zCDP composition
- ▶ Burstiness via renewal processes
- ▶ Real workload traces
- ▶ Joint budget–leakage bound
- ▶ Equivalence-checked semantic cache
- ▶ Multi-table / JOIN via residual sensitivity

> thank you _

questions ?

refik can öztaş // N25142279

repo: github.com/canoztas/cmp653-project

4,155 trials | 62 unit tests | 13-section paper